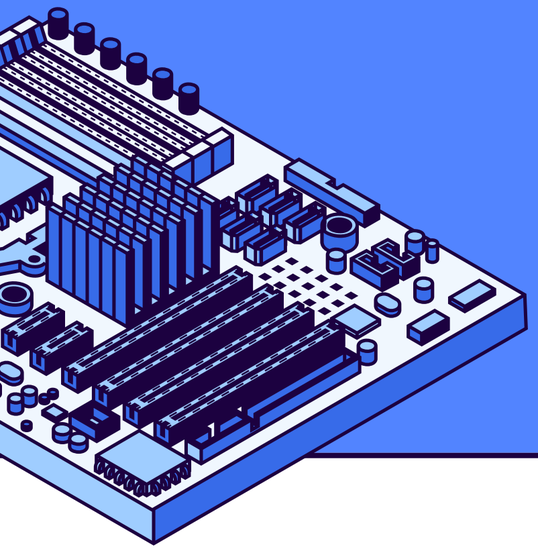
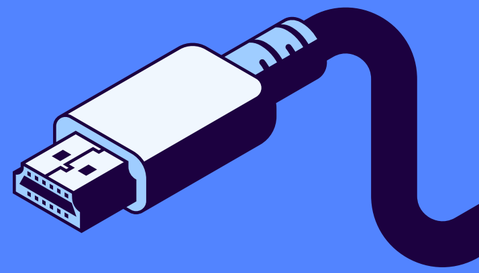
FIT3143
Applied 2





Shee Seng Cheng; 34612467; sshe0113@student.monash.edu
Tay Chee Hsian; 34612513; ctay0040@student.monash.edu

Task 1

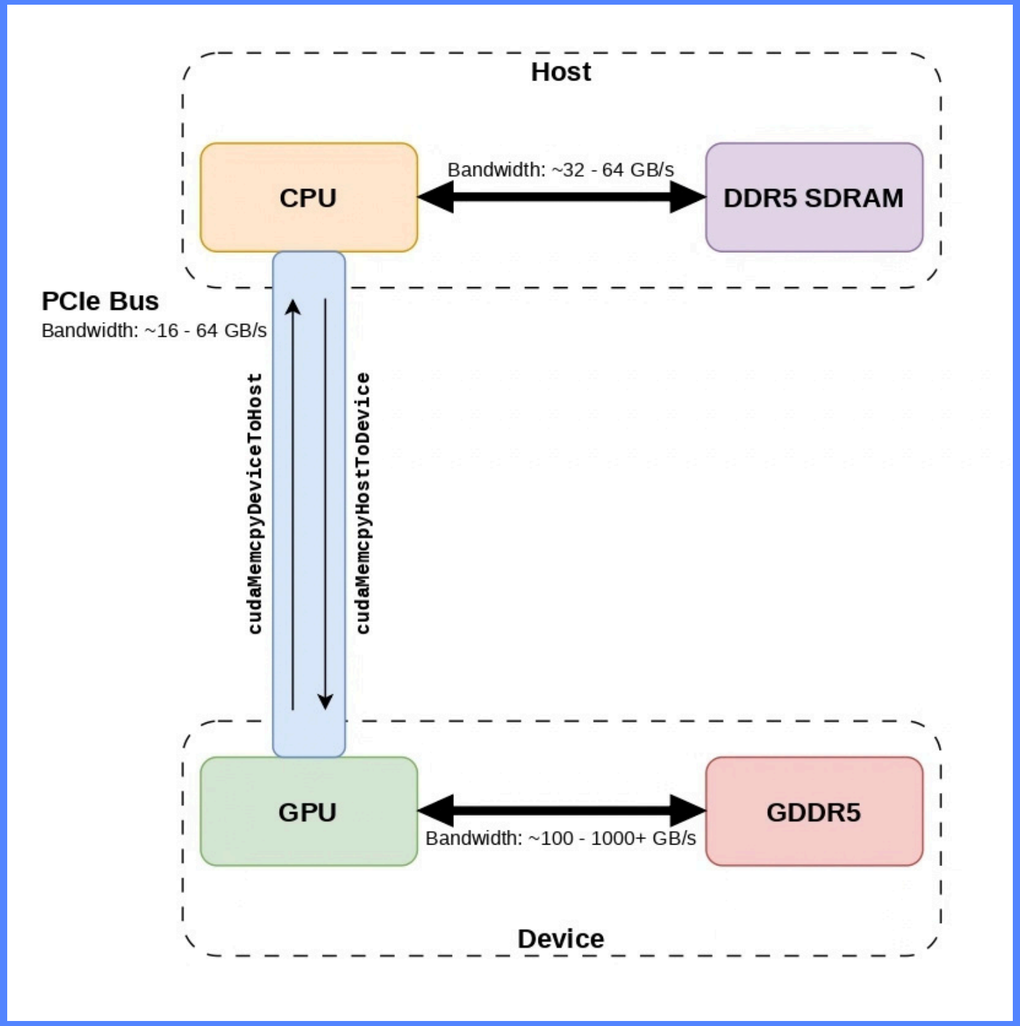
IMAGE ROTATION WITH GPU

- Image Data Transfer
- CUDA Programming Model
- GPUDirect Storage



Overview

- The CPU and GPU operate in physically distinct memory domains.
- The initial image sits in Host memory (DDR5 SDRAM), which is optimized for low latency and sequential CPU instructions.
- The GPU computation requires the image to reside in dedicated Device memory (GDDR5 VRAM), which is architected for massive data parallelism and high throughput.



Transfer Steps

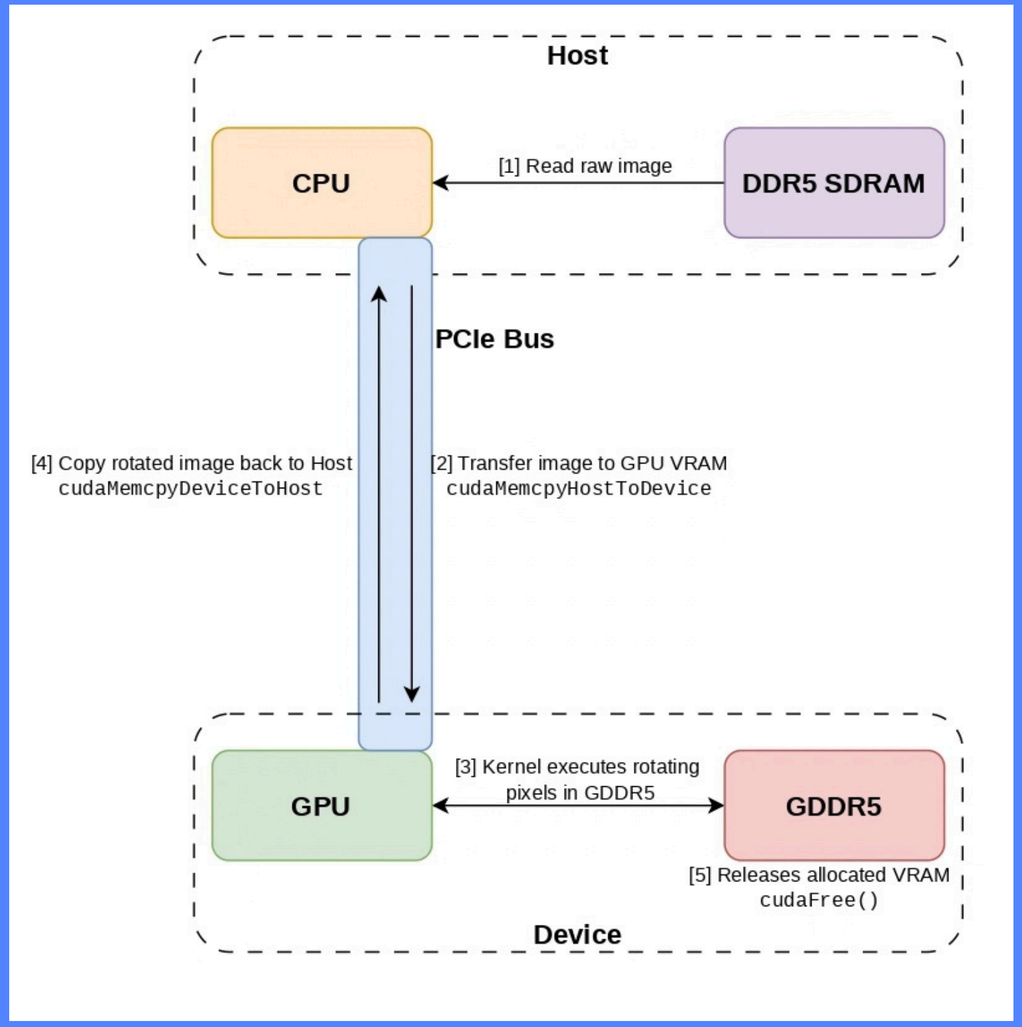
Step 1: The host calls cudaMalloc() to allocate a buffer in GPU GDDR memory

Step 2: The image travels from DDR5 across the PCIe bus into GDDR via cudaMemcpyHostToDevice

Step 3: The GPU executes the rotation kernel on the data inside GDDR

Step 4: The processed image travels back across PCIe into DDR5 via cudaMemcpyDeviceToHost so the CPU can save or display it

Step 5: The host calls cudaFree() to prevent GPU memory leaks



Responsibilities

Host (CPU)

```
// Allocate VRAM
cudaMalloc(&d_in, size);
cudaMalloc(&d_out, size);

// Transfer data to device
cudaMemcpy(d_in, h_in, size,
           cudaMemcpyHostToDevice);

// Launch kernel
rotateGPU<<<grid, block>>>(...);

// Transfer result back to host
cudaMemcpy(h_out, d_out, size,
           cudaMemcpyDeviceToHost);

// Free GPU resources
cudaFree(d_in);
cudaFree(d_out);
```

Device (GPU)

- Compute coprocessor tailored for Single Instruction, Multiple Threads (SIMT) workloads.
- Distributes thread block across physical hardware streaming multiprocessors.
- Thousands of lightweight threads run the same kernel concurrently, with each thread computing the trigonometric rotation matrix for its target pixel coordinates.

Execution Configuration

```
struct dim3 {unsigned int x; unsigned int y; unsigned int z;};
typedef struct dim3 dim3;

dim3 thread = {16, 16, 1};
dim3 block = {(width + 15)/16, (height + 15)/16, 1};

rotateGPU<<<grid, block>>>(d_in, d_out, width, height, cos_t, sin_t);
```

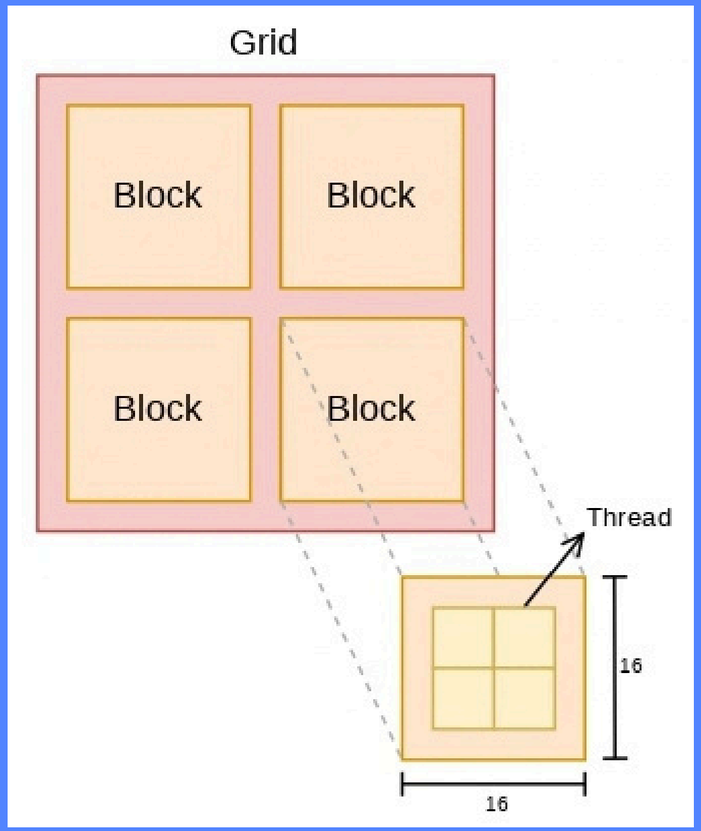
- Block groups individual threads into a local cooperative block scheduled onto a Streaming Multiprocessor.
- Grid arranges those blocks into a global grid that tiles the entire 2D image coordinate space.
- 256 threads per block guarantees execution in 8 full 32-thread hardware warps, maximizing SM occupancy and avoiding idle execution lanes.
- The CPU fires the kernel launch command to the GPU and continues immediately to its next line of code without waiting, letting CPU and GPU work at the same time.

Thread/Block Hierarchy

```
__global__ void rotateGPU(unsigned char *in, unsigned char
*out, int width, int height, float cos_t, float sin_t) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x < width && y < height) {
        int nx = (int)(x * cos_t - y * sin_t);
        int ny = (int)(x * sin_t + y * cos_t);

        if (nx >= 0 && nx < width && ny >= 0 && ny < height) {
            out[ny * width + nx] = in[y * width + x];
        }
    }
}
```



GPU Expected Speed-Up

MASSIVE ALU DENSITY

- CPU is built for low-latency serial execution; devotes most silicon to large caches and complex branch prediction with few powerful cores
- GPU is built for high-throughput data parallelism; devotes most silicon to thousands of ALUs, evaluating identical trigonometry rotation across thousands of pixels simultaneously.

MEMORY BANDWIDTH ADVANTAGE

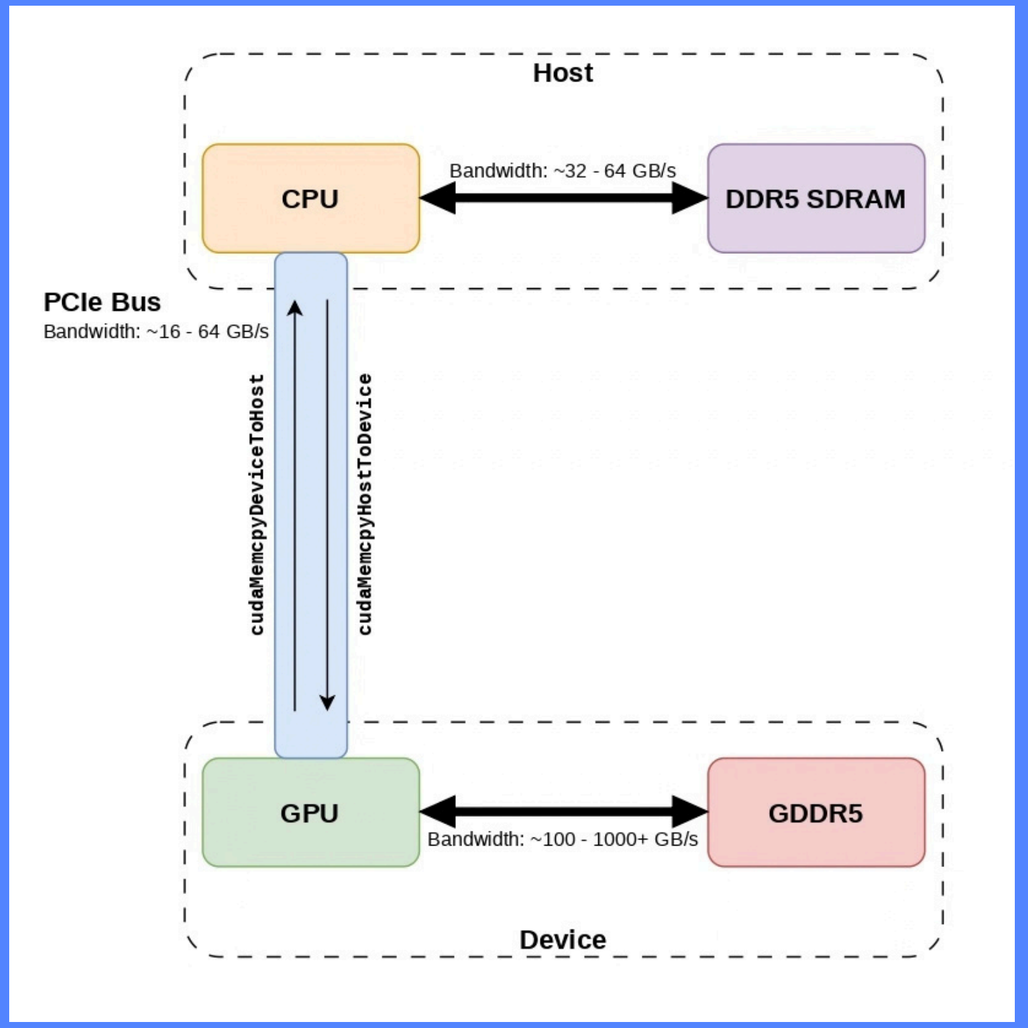
- CPU host system DDR5 RAM offers around 32 to 64 GB/s of memory bandwidth.
- GPU GDDR6 VRAM delivers around 100 to 1000+ GB/s across wide memory buses. Image rotation streams millions of bytes to and from DRAM, high memory bandwidth directly prevents thread starvation.

ZERO-OVERHEAD WARP LATENCY

- When a CPU thread waits for data from DRAM, it stalls or requires expensive OS context switching.
- When a GPU warp stalls waiting on global memory latency, the hardware Warp Scheduler instantly switches to another active, ready warp in the next clock cycle with zero overhead.

Bandwidth Disparity

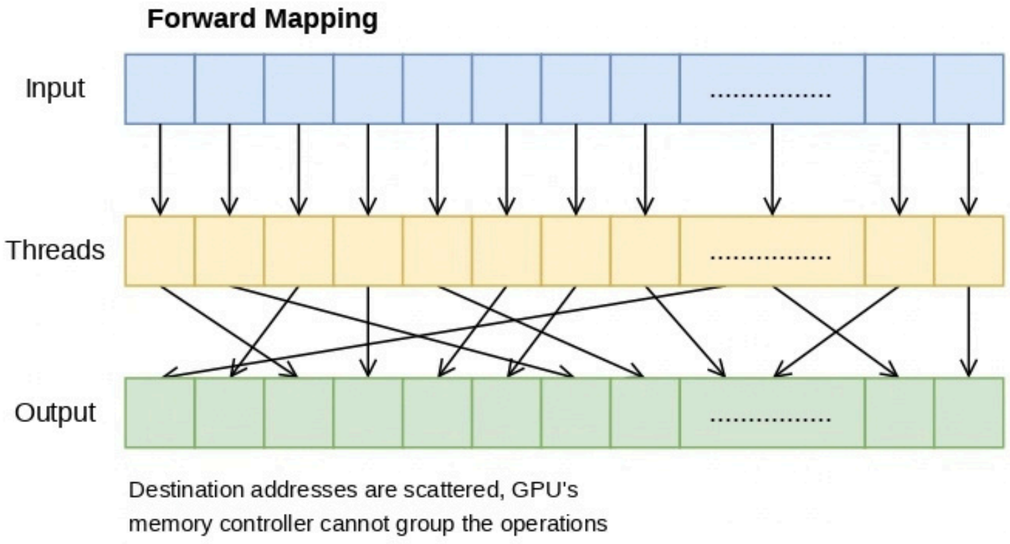
- The PCIe x16 bus (~16-64 GB/s) has significantly lower bandwidth than internal GPU memory (~100-1000+ GB/s).
- Moving large image arrays across the PCIe link often introduces greater latency than the actual rotation matrix calculations inside the GPU cores.
- Optimizing host-device data transfers is vital for high-throughput image processing pipelines.

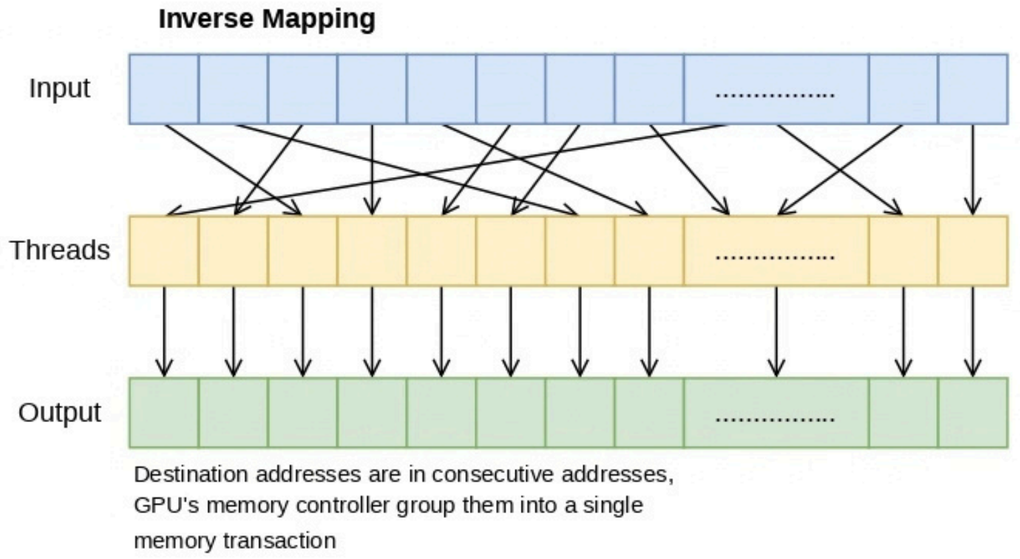


CUDA Programming Models

```
int nx = (int)(x * cos_t - y * sin_t);
int ny = (int)(x * sin_t + y * cos_t);
out[ny * w + nx] = in[y * w + x];
```

```
int sx = (int)(nx * cos_t + ny * sin_t);
int sy = (int)(-nx * sin_t + ny * cos_t);
out[ny * w + nx] = in[sy * w + sx];
```





CUDA Programming Models

```
int nx = (int)(x * cos_t - y * sin_t);
int ny = (int)(x * sin_t + y * cos_t);
out[ny * w + nx] = in[y * w + x];
```

(1) Thread Organization

Gives a baseline around 50x speed-up over CPU due to thousands of ALUs. However, the output writes are scattered (non-contiguous), causing uncoalesced global memory access which waste VRAM bandwidth.

```
int sx = (int)(nx * cos_t + ny * sin_t);
int sy = (int)(-nx * sin_t + ny * cos_t);
out[ny * w + nx] = in[sy * w + sx];
```

(2) Memory Optimization

Adjacent threads in a warp write to sequential memory addresses. The hardware combines 32 writes into a single transaction, boosting effective memory throughput and giving an additional 1.5x to 2x speed-up.

```
float cos_t = __cosf(angle);
float sin_t = __sinf(angle);
```

(3) Compute Optimization

For dynamic per-pixel angles, this bypasses slow multi-cycle software math functions, calculating trigonometric angles in hardware at the Special Function Unit (SFU) to maximize compute throughput.

GPUDirect Storage

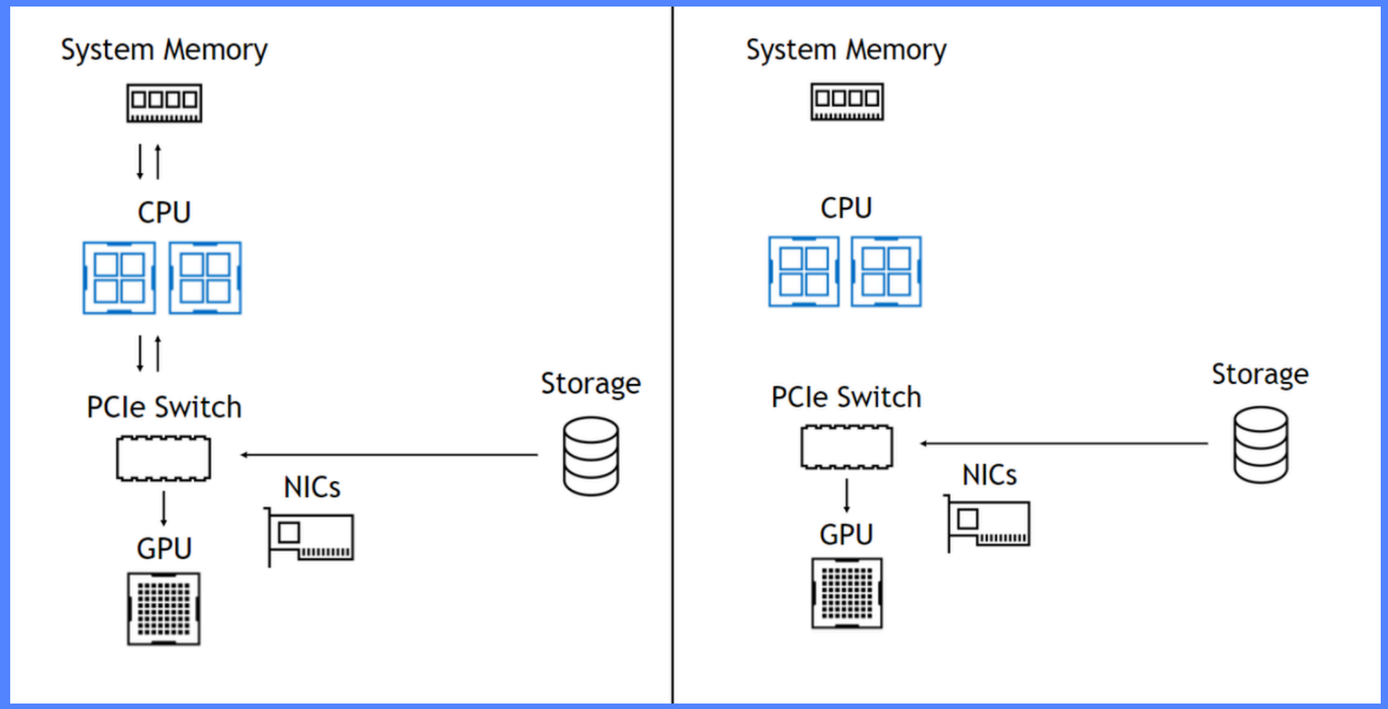


Image Source: Newburn et al. (2021)

Without GPUDirect Storage

- Traverses the PCIe bus twice, creating a throughput bottleneck.
- Stalls CPU cycles and consumes host memory bandwidth on intermediate buffer copies.

With GPUDirect Storage

- Eliminates redundant bounce buffers, freeing CPUc cores for concurrent tasks.
- Cuts host-to-device transit latency to feed GPU compute engines at full Non-Volatile Memory Express (NVMe) line rate.

GPUDirect Storage for Image Rotation

WHEN IT IS BENEFICIAL

- **Continuous High-Resolution Streaming:** GDS is beneficial when continuously streaming large raw images from fast NVMe storage to the GPU.
- **Prevents GPU Thread Starvation:** Faster data transfer helps keep GPU threads supplied with data instead of waiting for storage I/O.
- **CPU Offloading & Latency Reduction:** GDS reduces CPU involvement and unnecessary data copies between NVMe, RAM, and CPU memory.

WHEN IT IS NOT/LIMITED BENEFICIAL

- **Compressed File Formats:** JPEG/PNG images still require CPU-side decompression before the GPU can process them.
- **Pre-Loaded In-Memory Workloads:** GDS provides little benefit when images are already loaded in RAM because there is no disk I/O to optimize.
- **Compute-Bound Single Images:** GDS provides limited speed-up when GPU computation dominates the processing time rather than data transfer.

Task 2

ETHICAL IMPLICATIONS OF SCALING HPC FOR AI

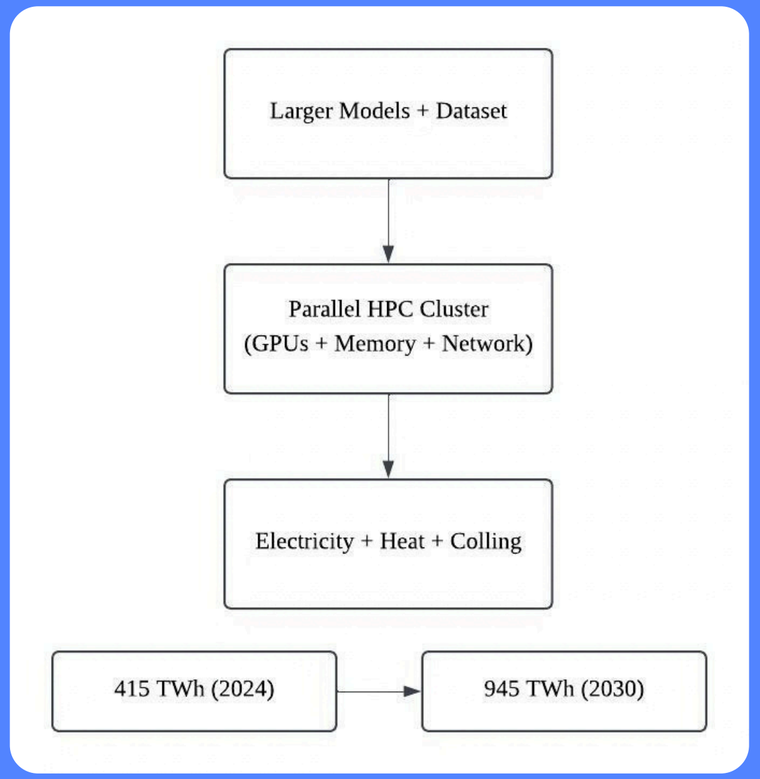
- Scaling must account for environmental cost
- Data and security risks
- Unequal access to compute
- Concentration of technological power



AI Scale Requires HPC Scale

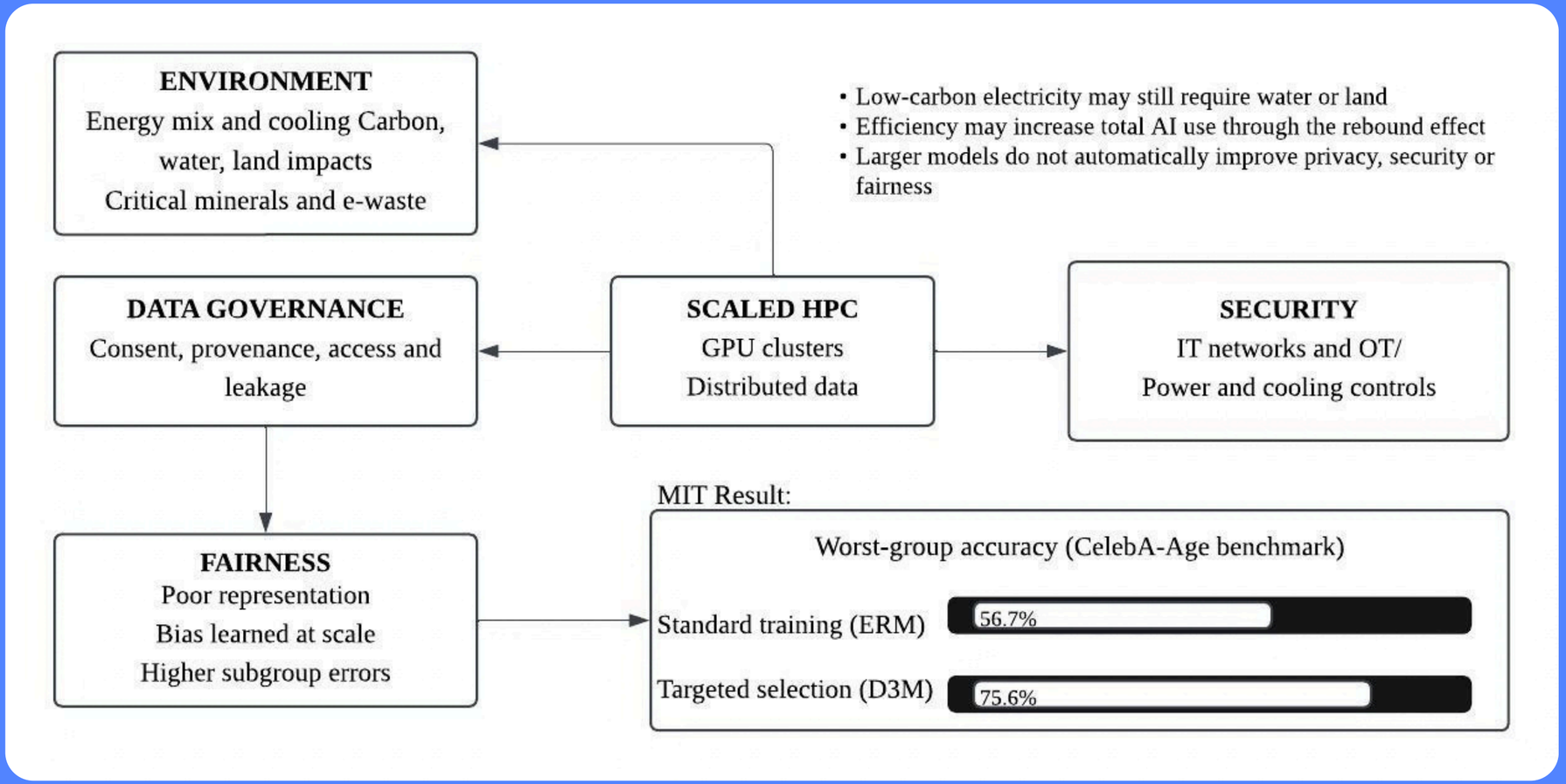
- Larger models and datasets require parallel GPU clusters
- **HPC also requires memory, networking, electricity and cooling**
- Data centres: **415 TWh in 2024 (1.5% global electricity) → 945 TWh by 2030**
- Geographical concentration **creates local grid and water pressure**

AI growth converts software demand into physical infrastructure demand.



Source: Kandemir (2025); IEA (2025); Aczel et al. (2026)

Connected Risks of HPC Scaling

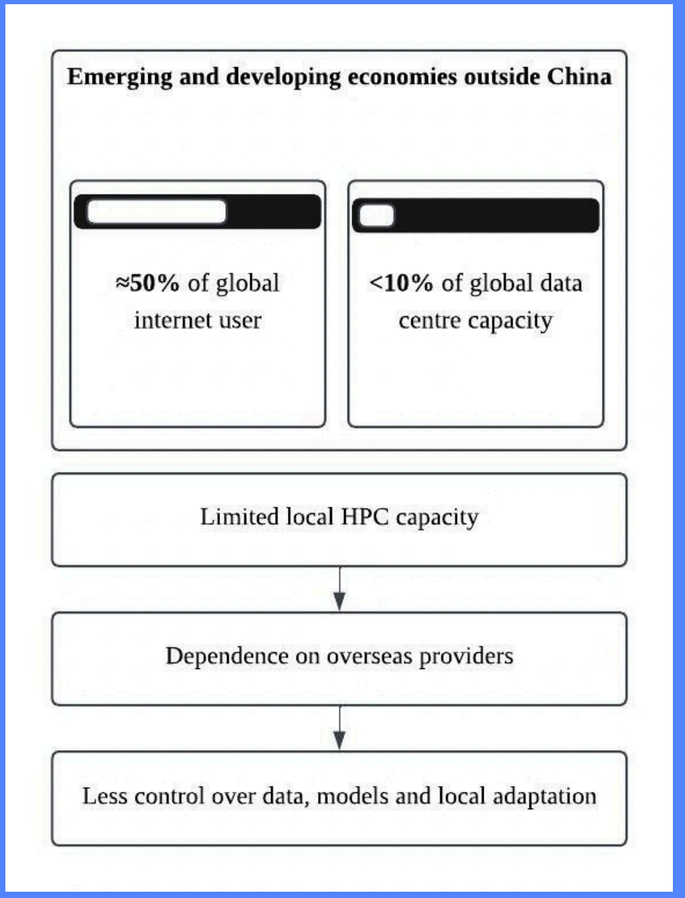


Source: Aczel et al. (2026); Gomstyn and Jonker (2026); Partner et al. (2026); Jain et al. (2024)

Unequal Access to HPC Resources

- HPC requires expensive **hardware, energy, cooling and expertise**
- Limited local compute can **increase dependence** on overseas providers
- Communities **may** carry infrastructure burdens without receiving equal AI benefits

Access to an AI service does not equal access to the compute needed to build or adapt AI.



Sources: IEA (2025); Aczel et al. (2026); Kandemir (2025)

A Framework for Responsible HPC

- **Measure and limit resource use:** Report carbon, water and land footprints; use fit-for-purpose models and resource budgets.
- **Protect data and test fairness:** Apply consent, data minimisation and access controls; evaluate model performance across subgroups.
- **Share access and benefits:** Support compute access for smaller institutions and involve communities in data-centre planning.

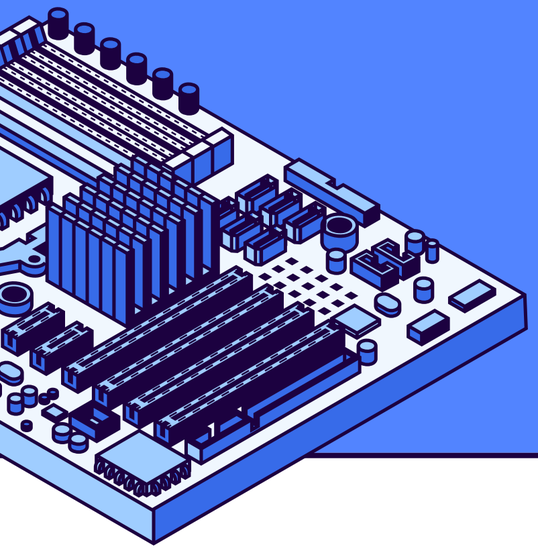
Efficiency per task may improve while total resource use continues to grow.

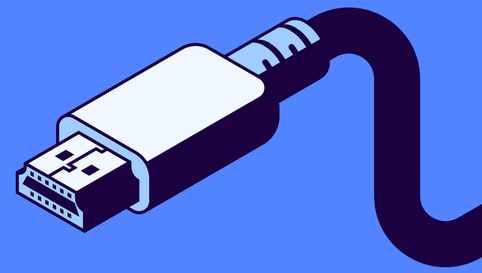
Proposed framework informed by IEA (2025);Aczel et al. (2026); Kandemir (2025);Gomstyn and Jonker (2026);Partner et al. (2026);Jain et al. (2024).

Conclusion

- GPU acceleration depends on parallelism, efficient memory access and controlled data-transfer overhead.
- Scaling HPC also creates environmental, governance, security, fairness and accessibility challenges.
- Future work should benchmark end-to-end GPU performance and measure the wider impacts of HPC deployment.

Thank You





References

Aczel, M., Chamanara, S., Matin, M., Farsi, A., Marwala, T, & Madani, K. (2026). Environmental cost of artificial intelligence: Carbon, water, and land footprints. United Nations University. *https://unu.edu/inweh/collection/environmental-cost-of-AIs-Enrgy-Use-Carbon-water-and-land-footprints*

Gomstyn, A., & Jonker, A. (2026). Exploring privacy issues in the age of AI. IBM. *https://www.ibm.com/think/insights/ai-privacy*

IEA. (2025). Energy and AI – analysis - IEA. IEA. *https://www.iea.org/reports/energy-and-ai*

Jain, S., Hamidieh, K., Georgiev, K., Ilyas, A., Ghassemi, M., & Mądry, A. (2024). Data debiasing with datamodels (D3M): Improving subgroup robustness via data selection. *https://arxiv.org/pdf/2406.16846*

Kandemir, M. (2025). Why AI uses so much energy—And what we can do about it. Institute of Energy and the Environment. *https://iee.psu.edu/news/blog/why-ai-uses-so-much-energy-and-what-we-can-do-about-it*

References

Newburn, C., Modukuri, K. K., & Datta, K. (2021, August 23). Accelerating IO in the modern data center: Magnum IO storage. NVIDIA Technical Blog. *https://developer.nvidia.com/blog/accelerating-io-in-the-modern-data-center-magnum-io-storage/*

Partner, A. R., York, N., & Catharina Glugla Partner, D. (2026). Why data centers are high value targets for cyber attacks. A&O Shearman. *https://www.aoshearman.com/en/insights/data-center-insights/why-data-centers-are-high-value-targets-for-cyber-attacks*

Zewe, A. (2024). Researchers reduce bias in AI models while preserving or improving accuracy. MIT News | Massachusetts Institute of Technology. *https://news.mit.edu/2024/researchers-reduce-bias-ai-models-while-preserving-improving-accuracy-1211*